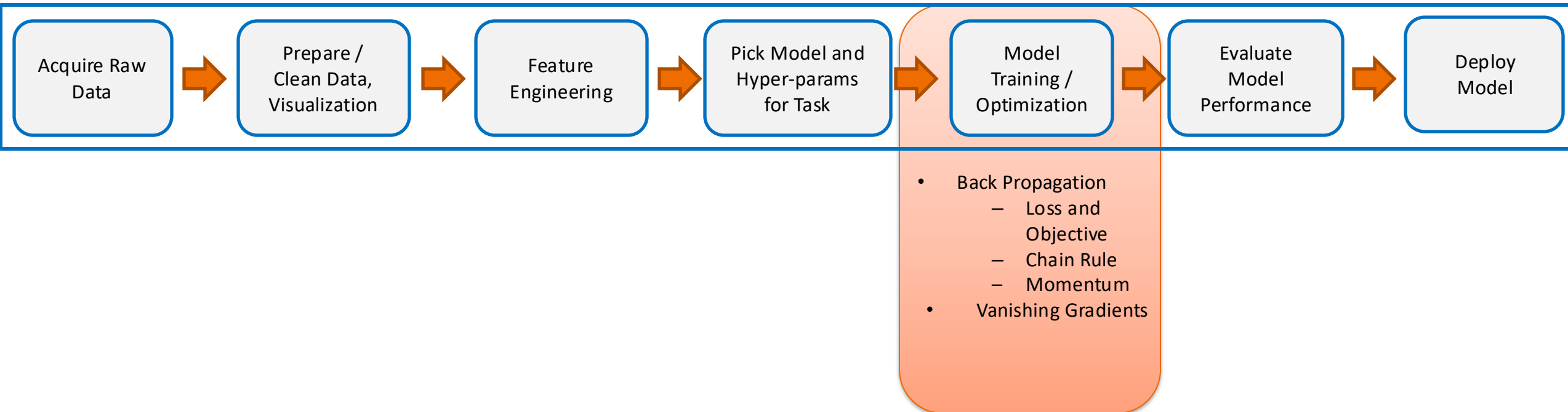


Focus for this lecture



Back Propagation

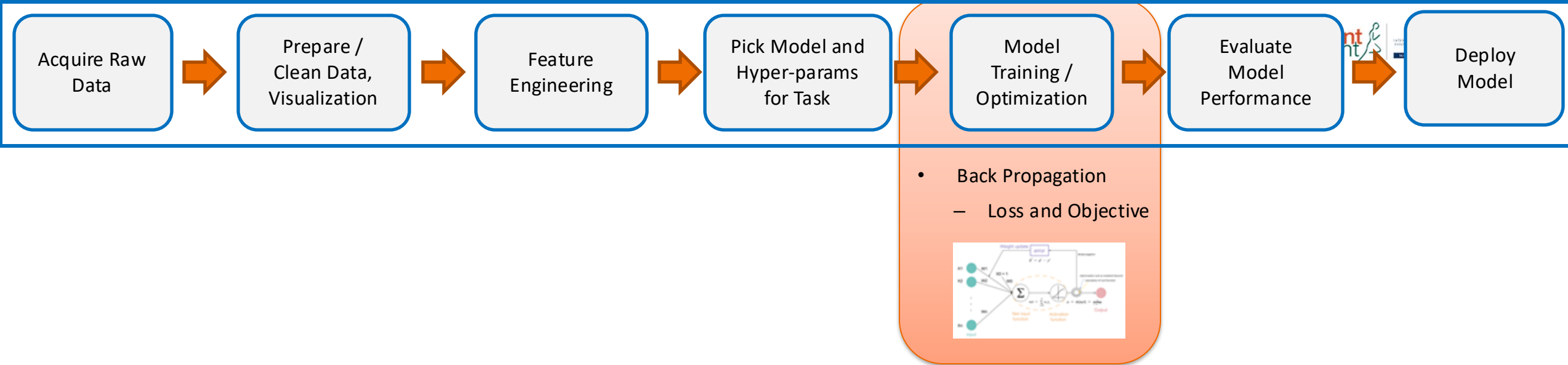
Training Neural Networks

Blank Slide: Recap of Gradient Descent

Blank Slide: Recap of Gradient Descent

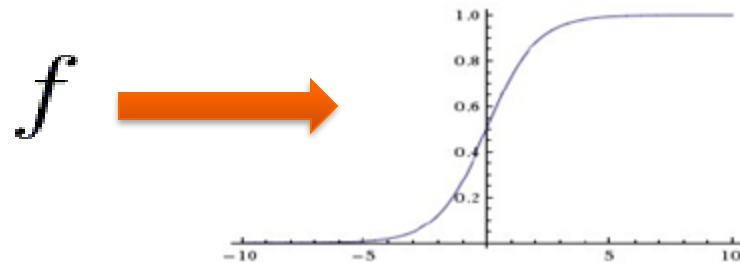
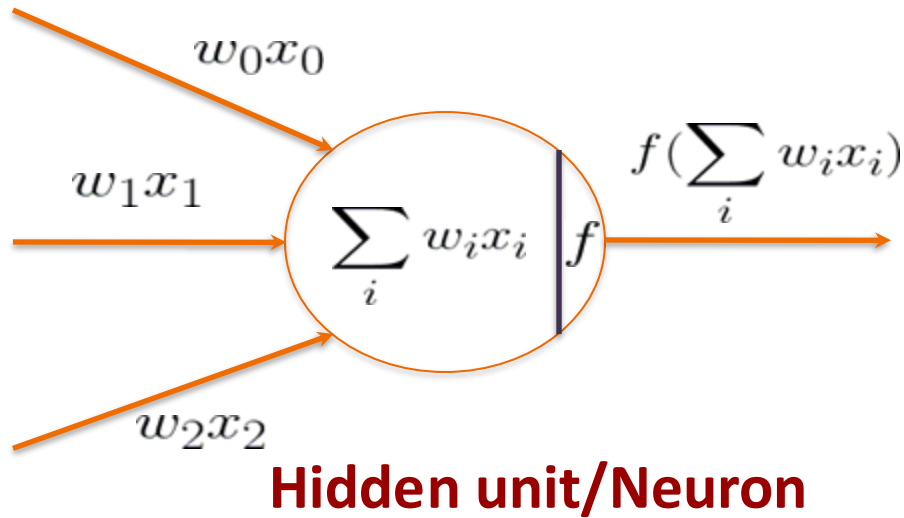
Plan for the Lecture

- Introduce the “Classical” Back propagation algorithm
 - “Error” back propagation
 - The same algorithm for all “deep” networks
 - Many practical refinements/tricks in implementation (later lecture)

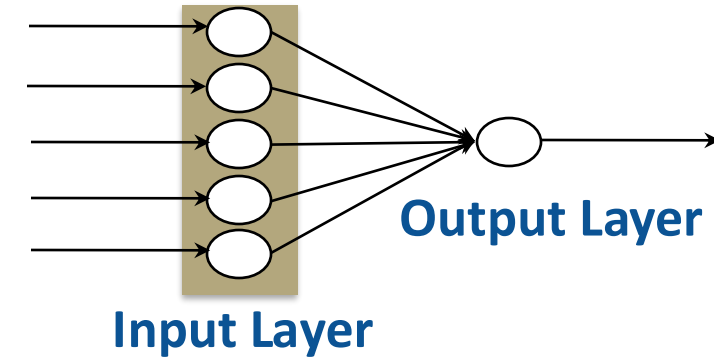


Loss/Objective, Error and Learning

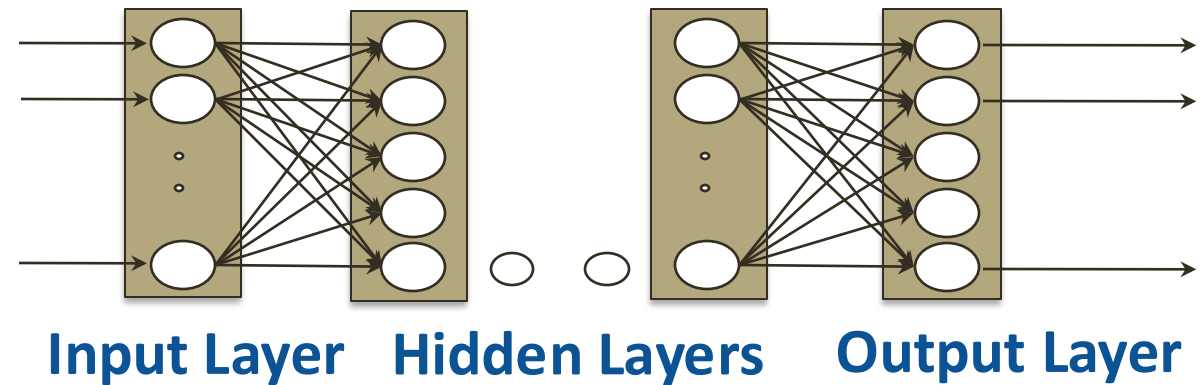
Neuron, Perceptron and MLP



E.g. Sigmoid Activation Function

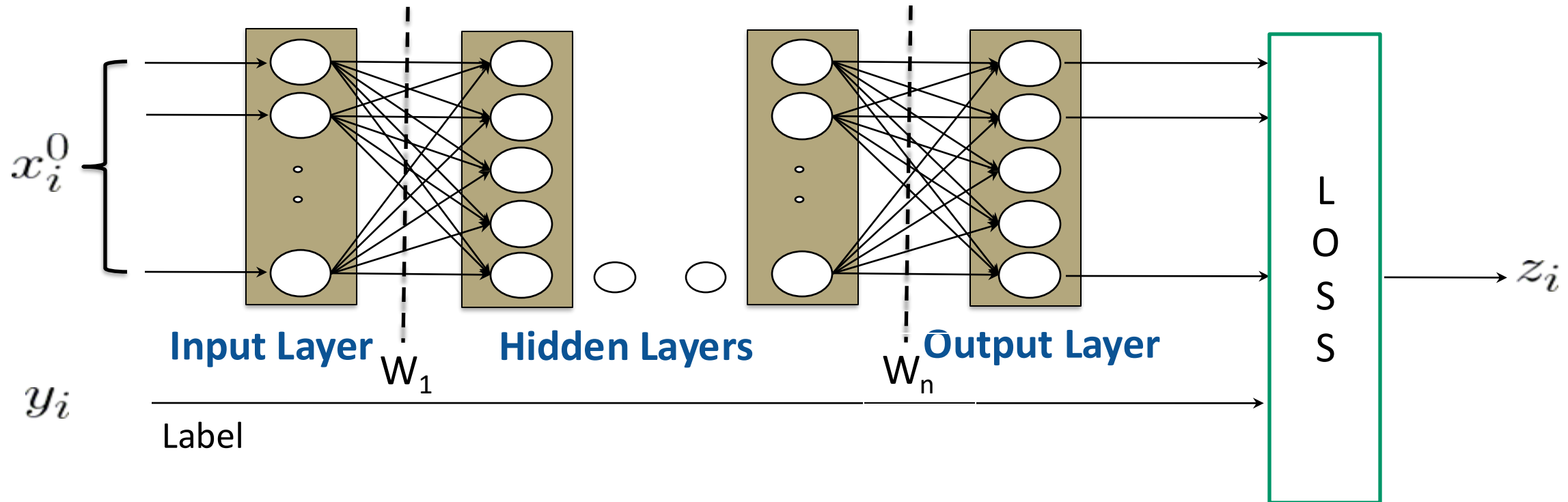


Perceptron



Multi Layer Perceptron

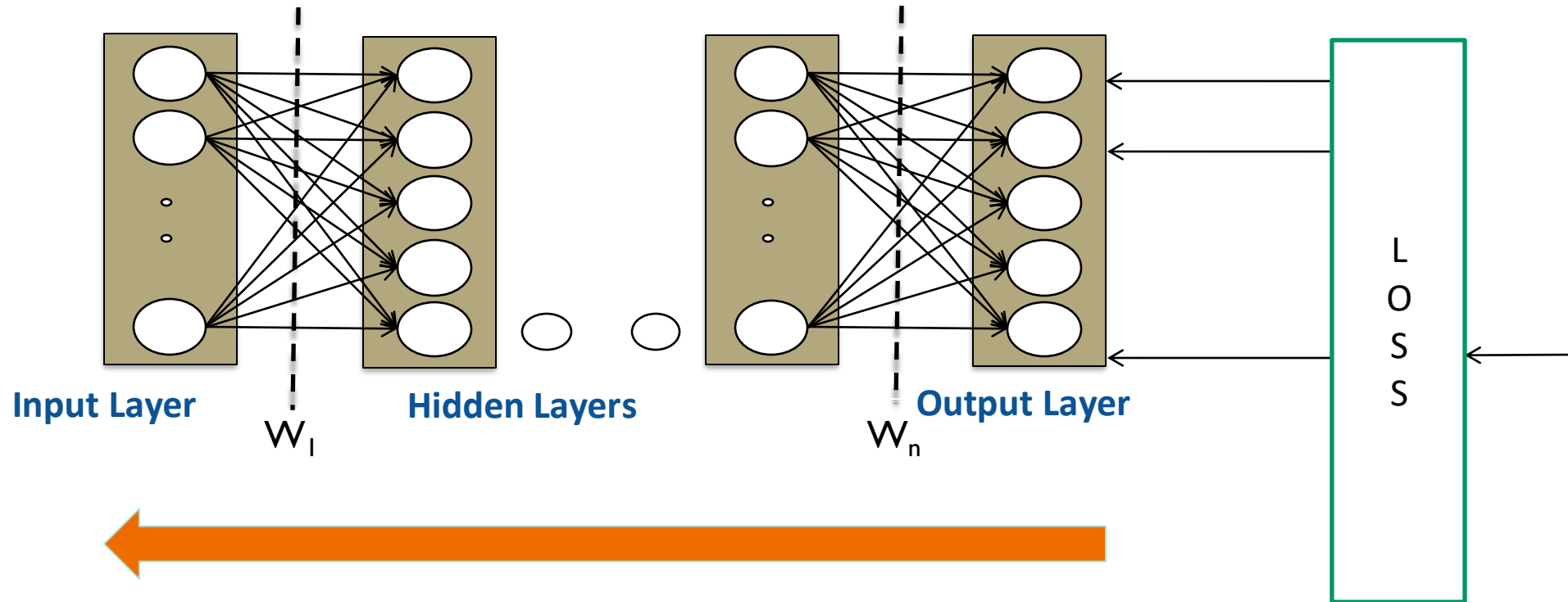
Loss or Objective



Objective: Find out the best parameters which will minimize the loss.

$$W^* = \arg \min_W \sum_{i=1}^N L(x_i^n, y_i; W) \longrightarrow \text{Weight vector}$$
$$z_i = \frac{1}{2} \| x_i^n - y_i \|_2^2 \quad \text{E.g. Squared Loss}$$

Back Propagation

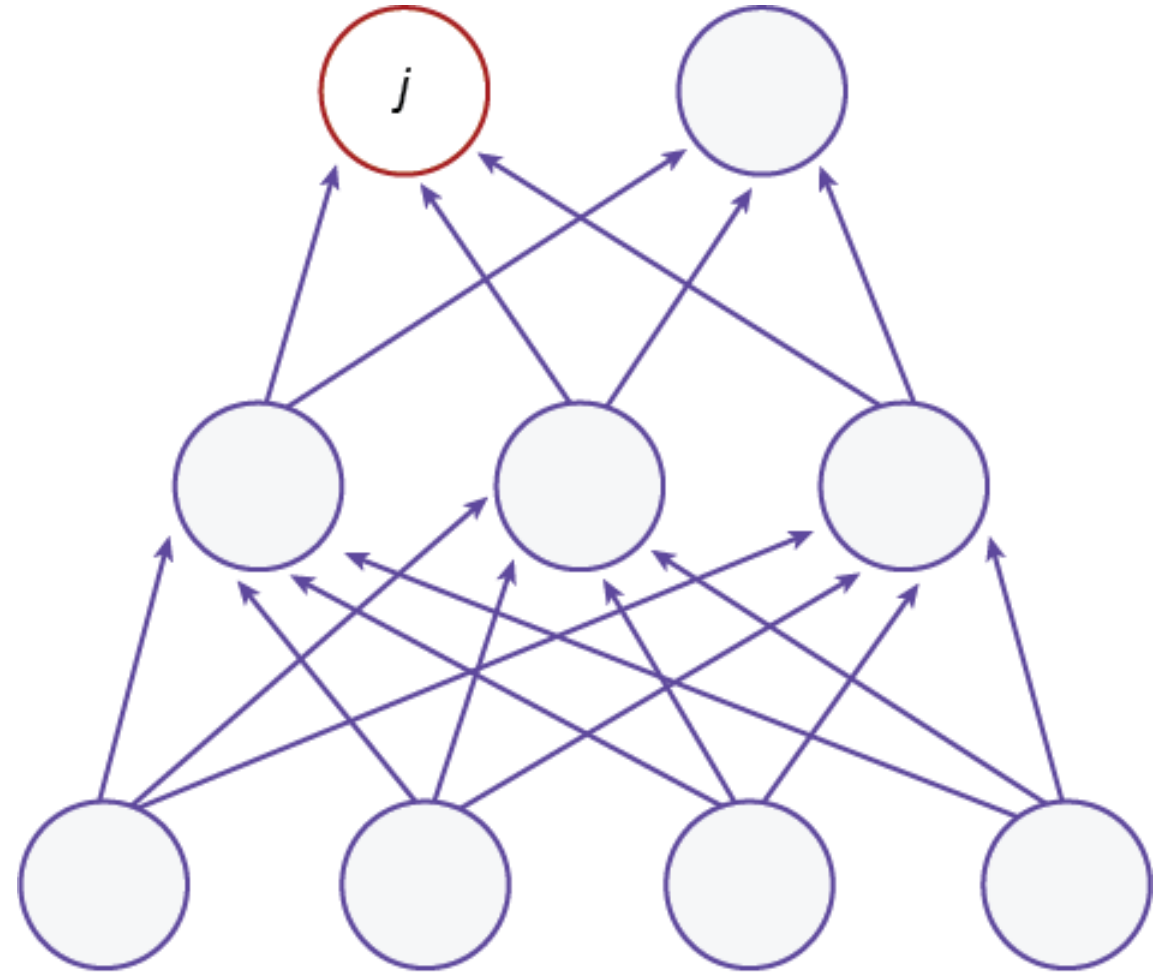


Solution: Iteratively update W along the direction where loss decreases.

Each layer's weights are updated based on the derivative of its output w.r.t. input and weights

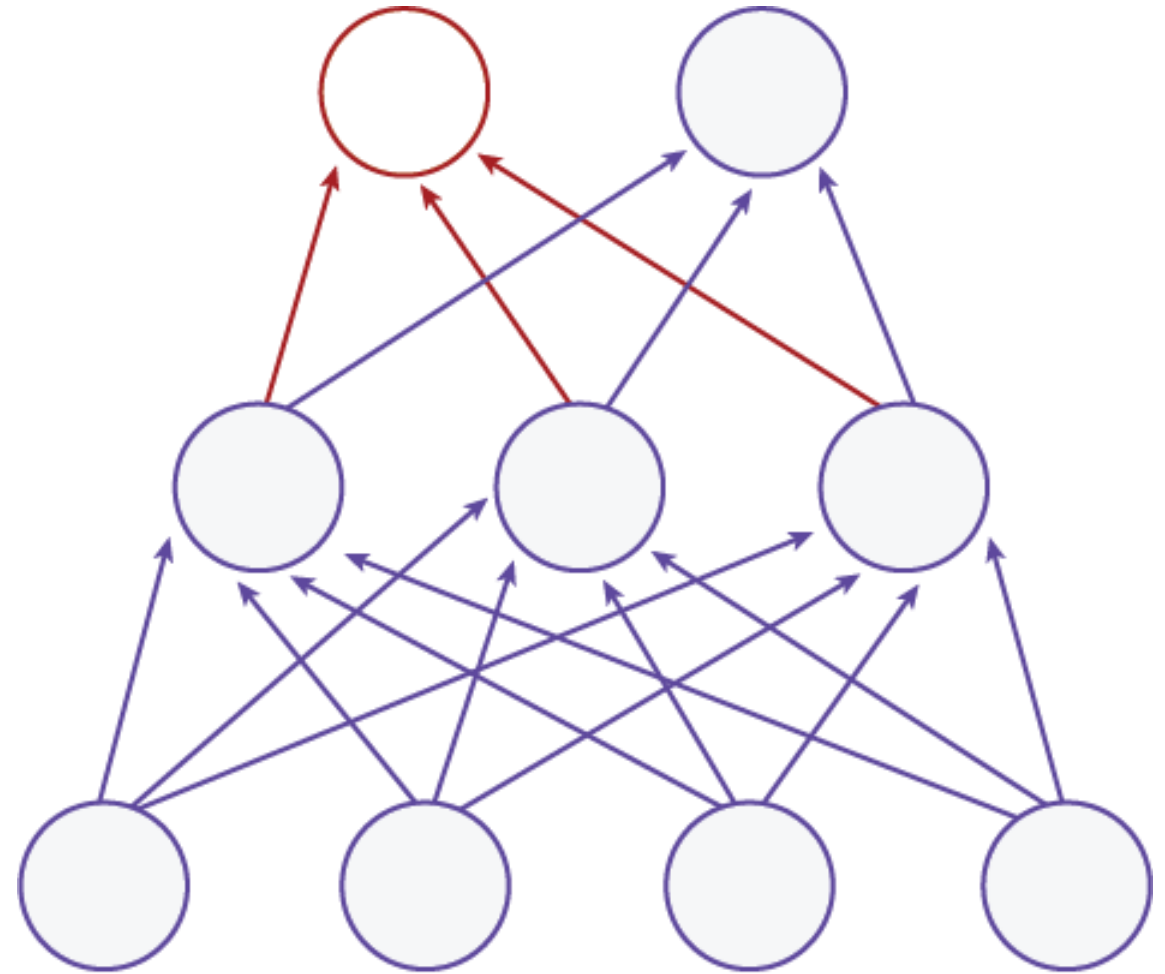
Error Back Propagation

- Calculate error



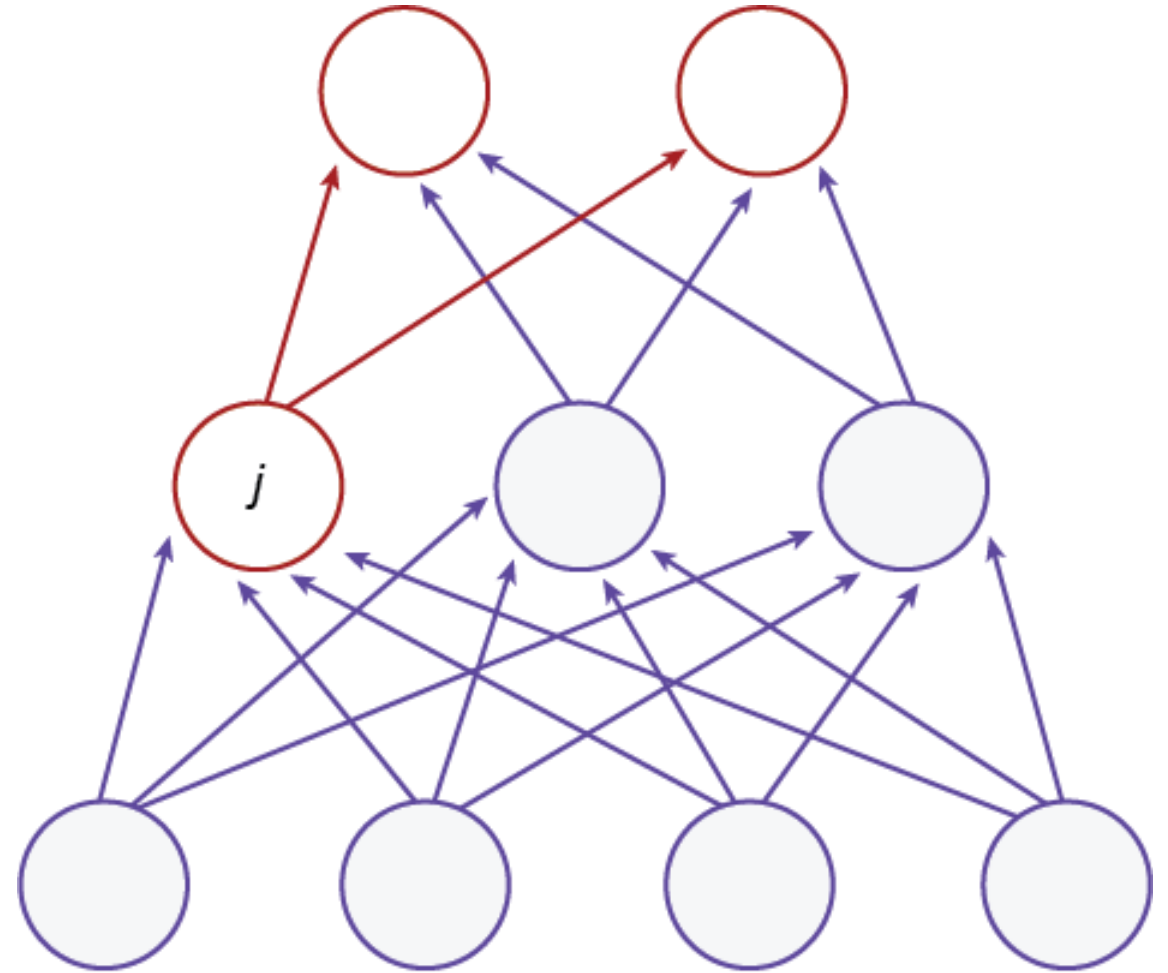
Error Back Propagation

- Determine updates for weights going to outputs.



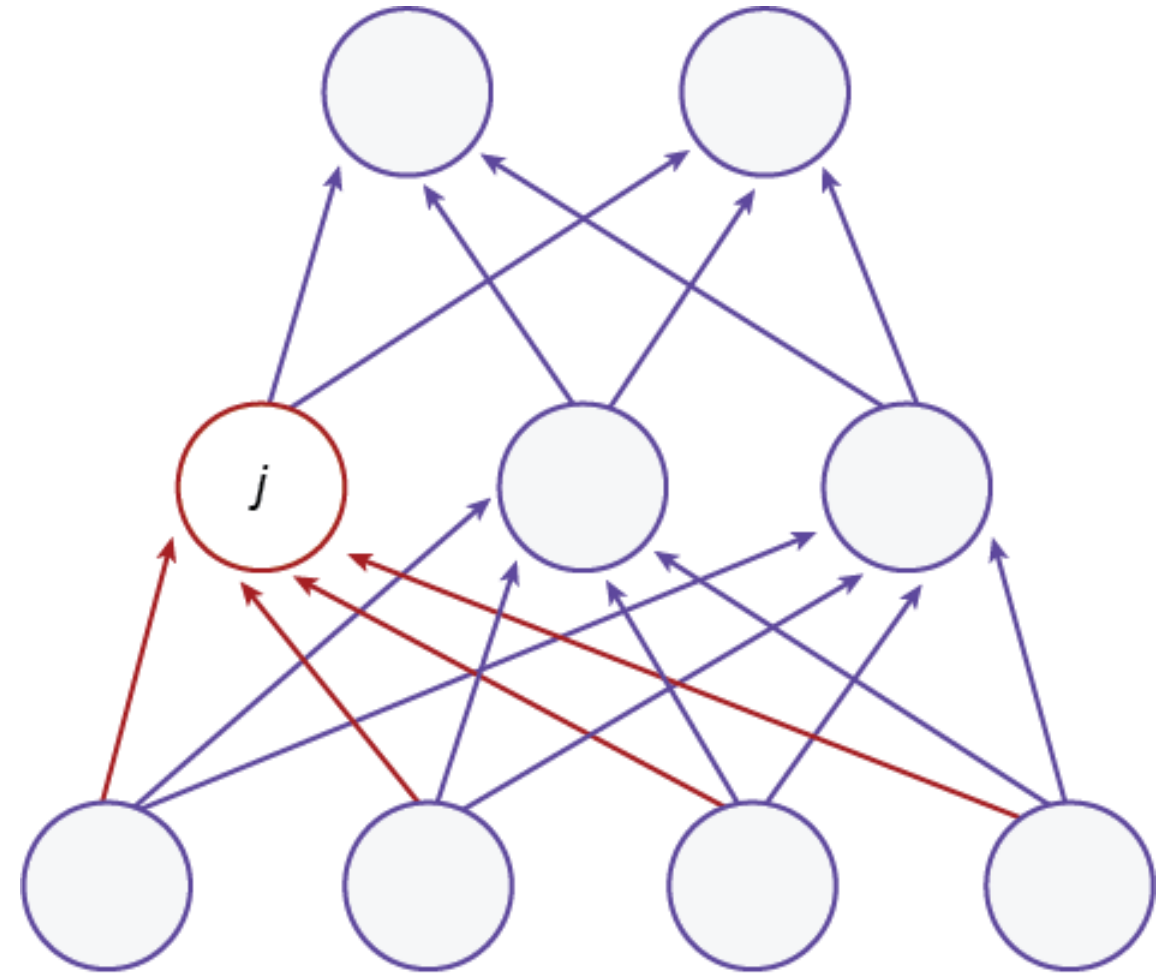
Error Back Propagation

- Calculate error for hidden units



Error Back Propagation

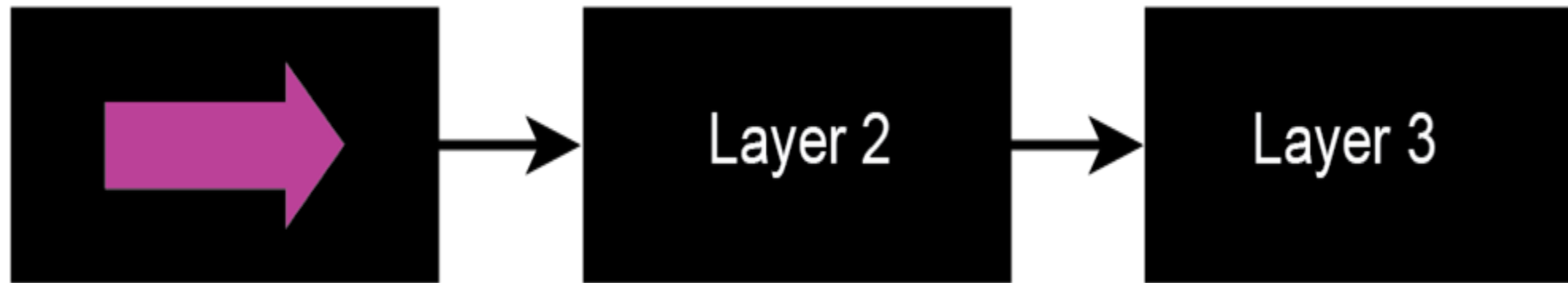
- Determine updates for weights to hidden units using hidden-unit errors.



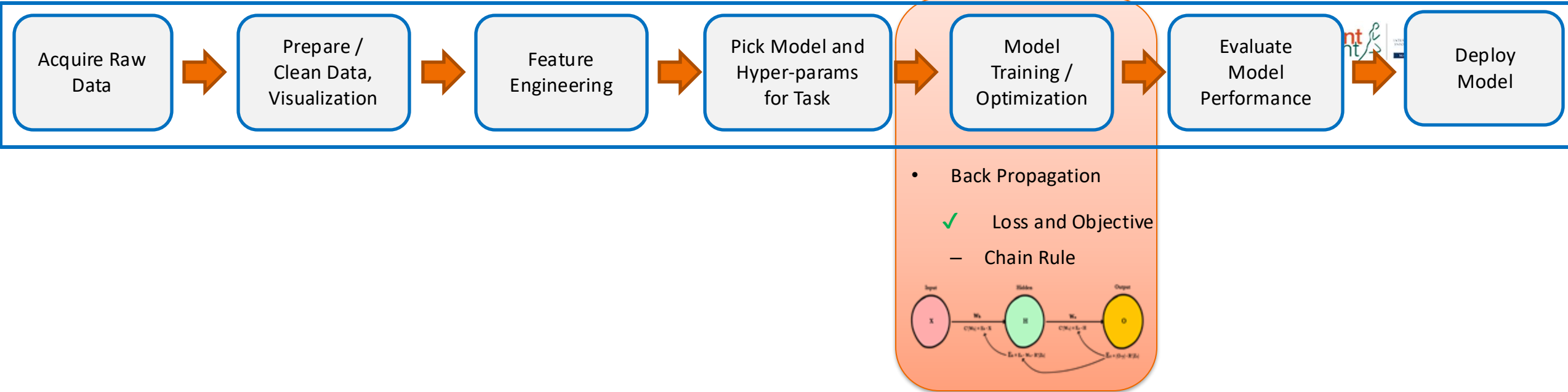
BLANK SLIDE

Neural Network Training

- **Step 1:** Compute loss on mini-batch [F-Pass]
- **Step 2:** Compute gradients w.r.t parameters[B-Pass]

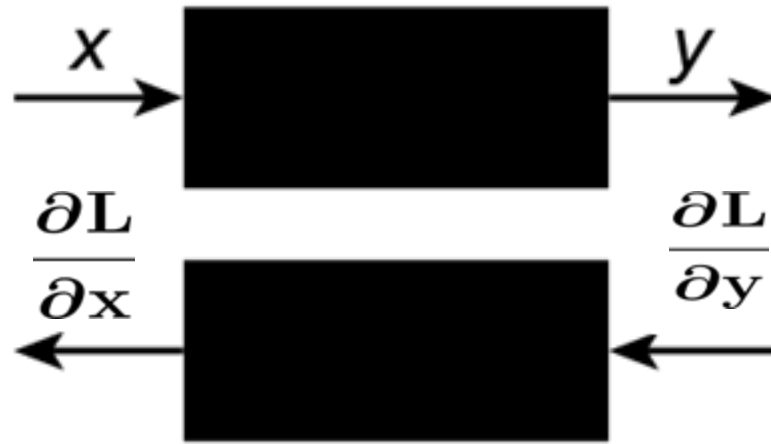


$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$



Chain Rule

Chain Rule



Given $y(x)$ and $\frac{\partial L}{\partial y}$
What is $\frac{\partial L}{\partial x}$?


$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}$$

Chain Rule



Given $\frac{\partial y}{\partial x}$ and $\frac{\partial L}{\partial y}$



$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}$$

What is $\frac{\partial L}{\partial x}$?

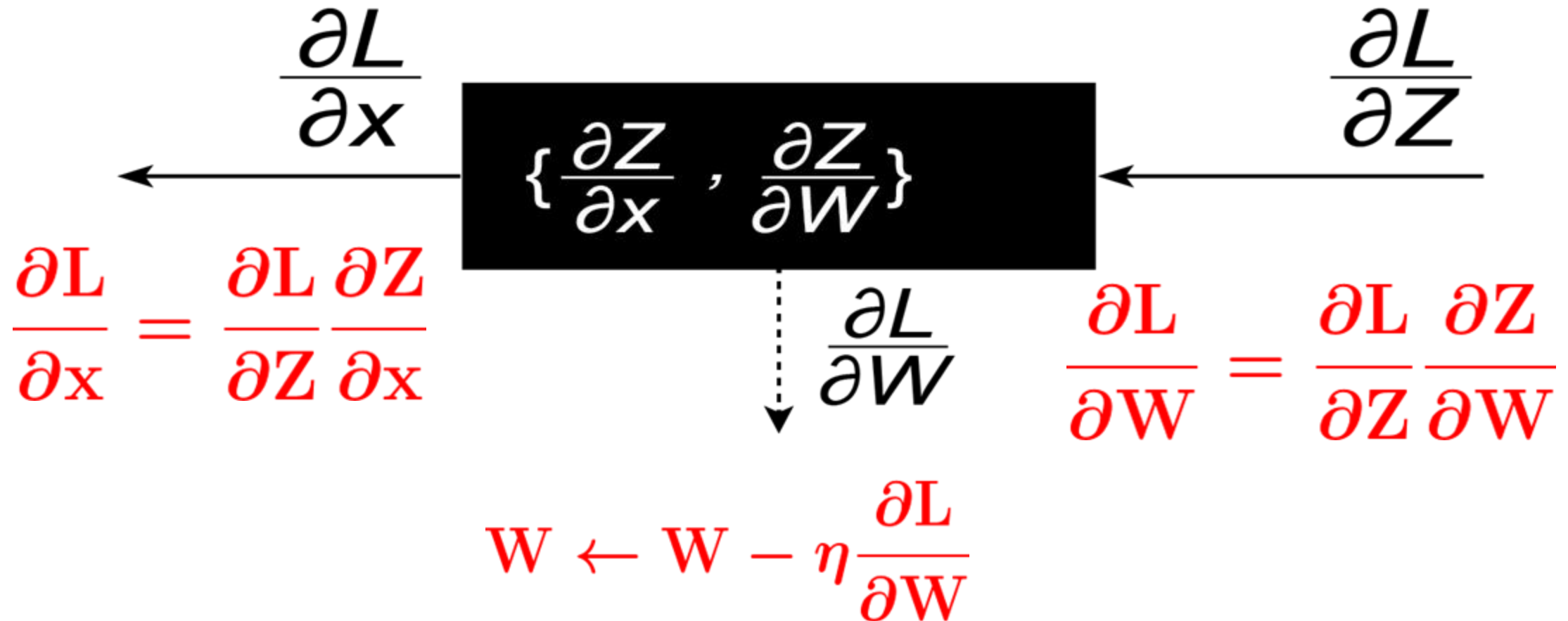
For each block/parameters, we only need to find $\frac{\partial y}{\partial x}$

Key Computation: Forward-Propagation



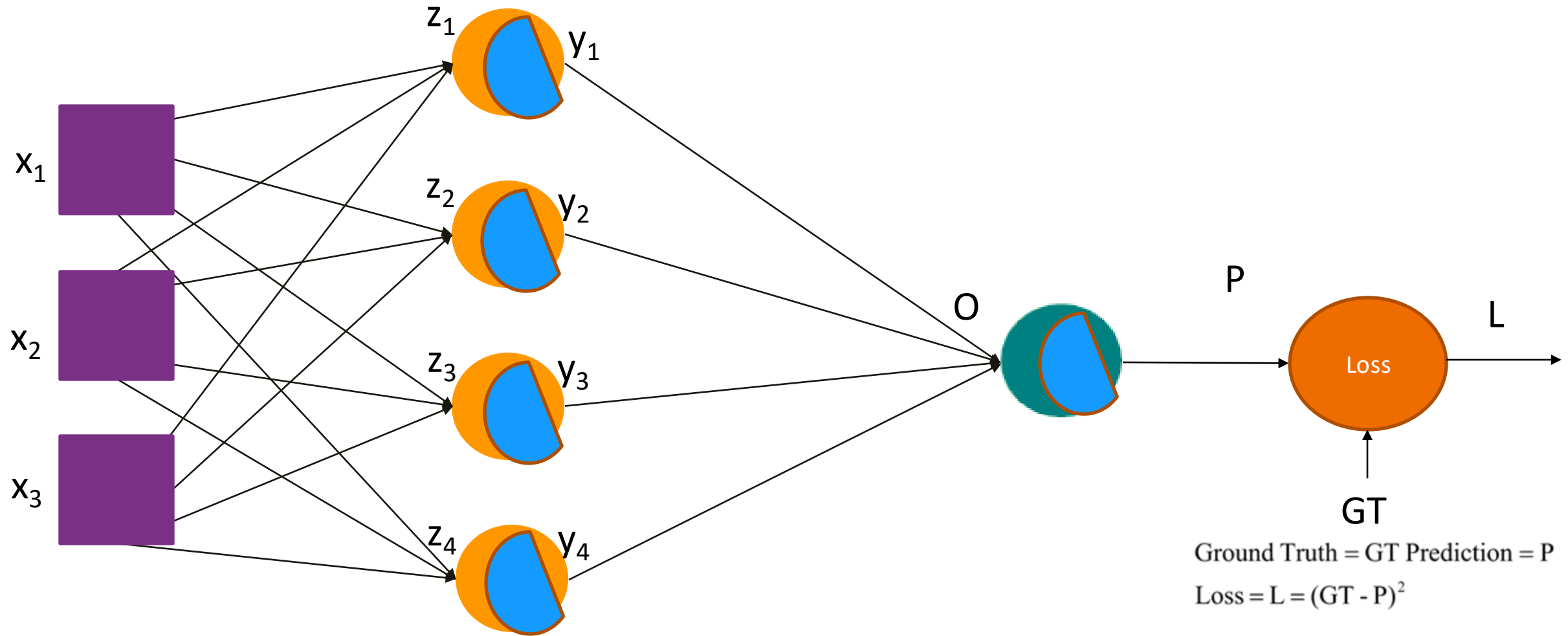
W is the parameter, say the weights within the box.

Key Computation: Backward-Propagation



BLANK SLIDE

Detailed View: Back Propagation



$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} \phi(z_1) \\ \phi(z_2) \\ \phi(z_3) \\ \phi(z_4) \end{bmatrix}$$

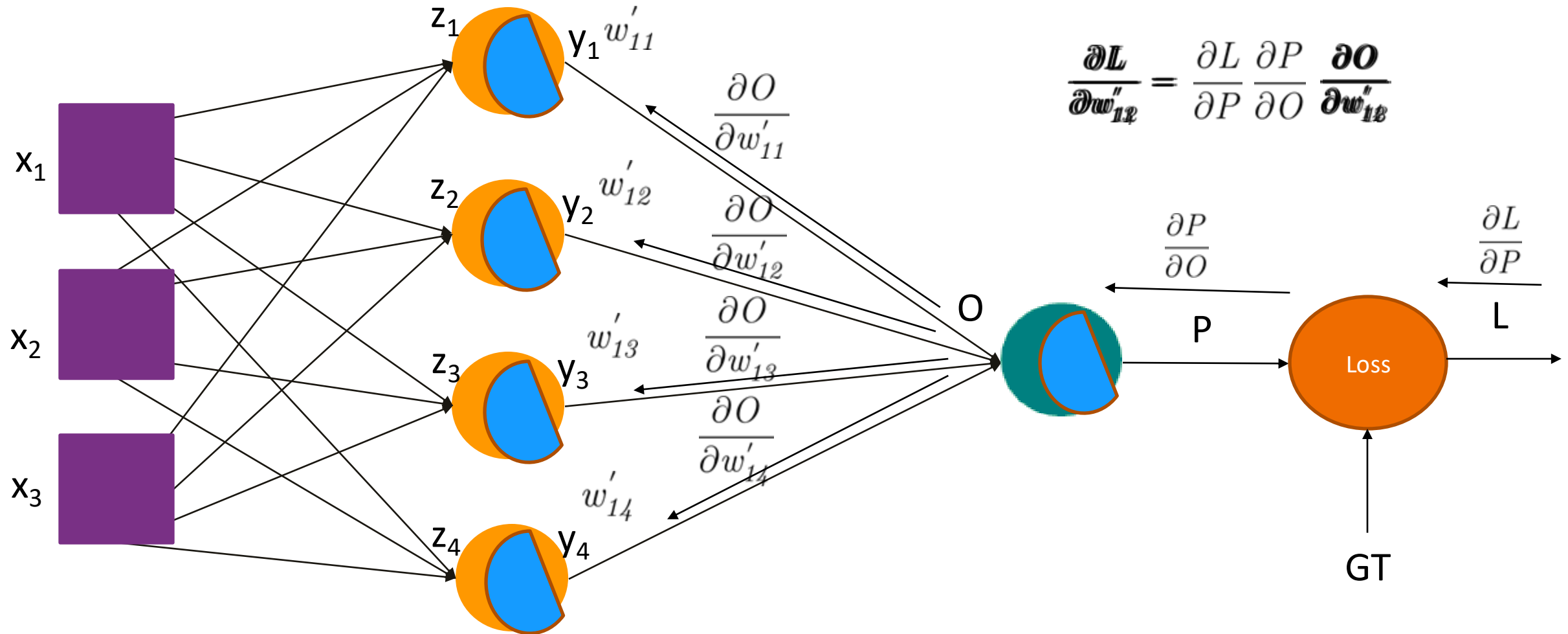
$$O = [w'_{11} \quad w'_{12} \quad w'_{13} \quad w'_{14}] \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}$$

$$P = \phi(O)$$

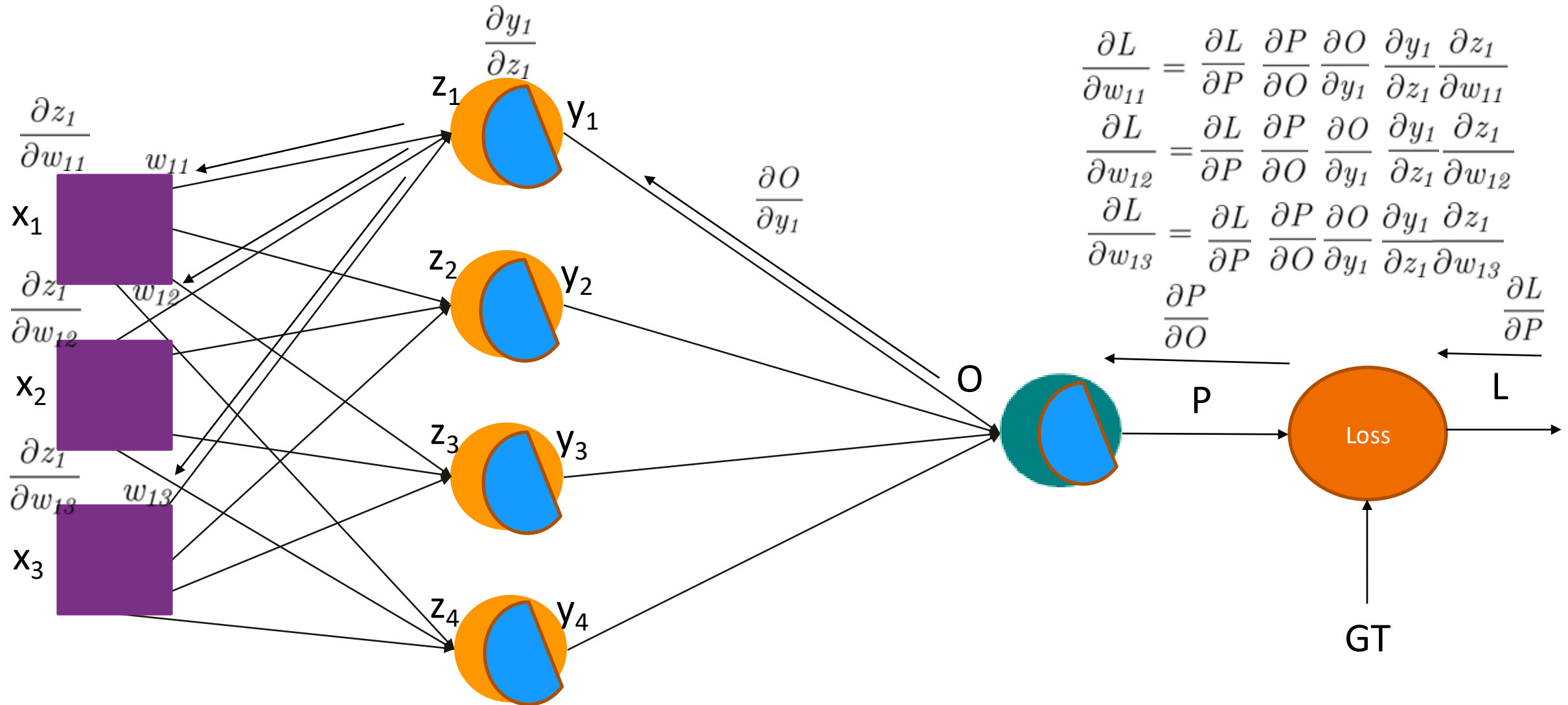
Ground Truth = GT Prediction = P

$$\text{Loss} = L = (GT - P)^2$$

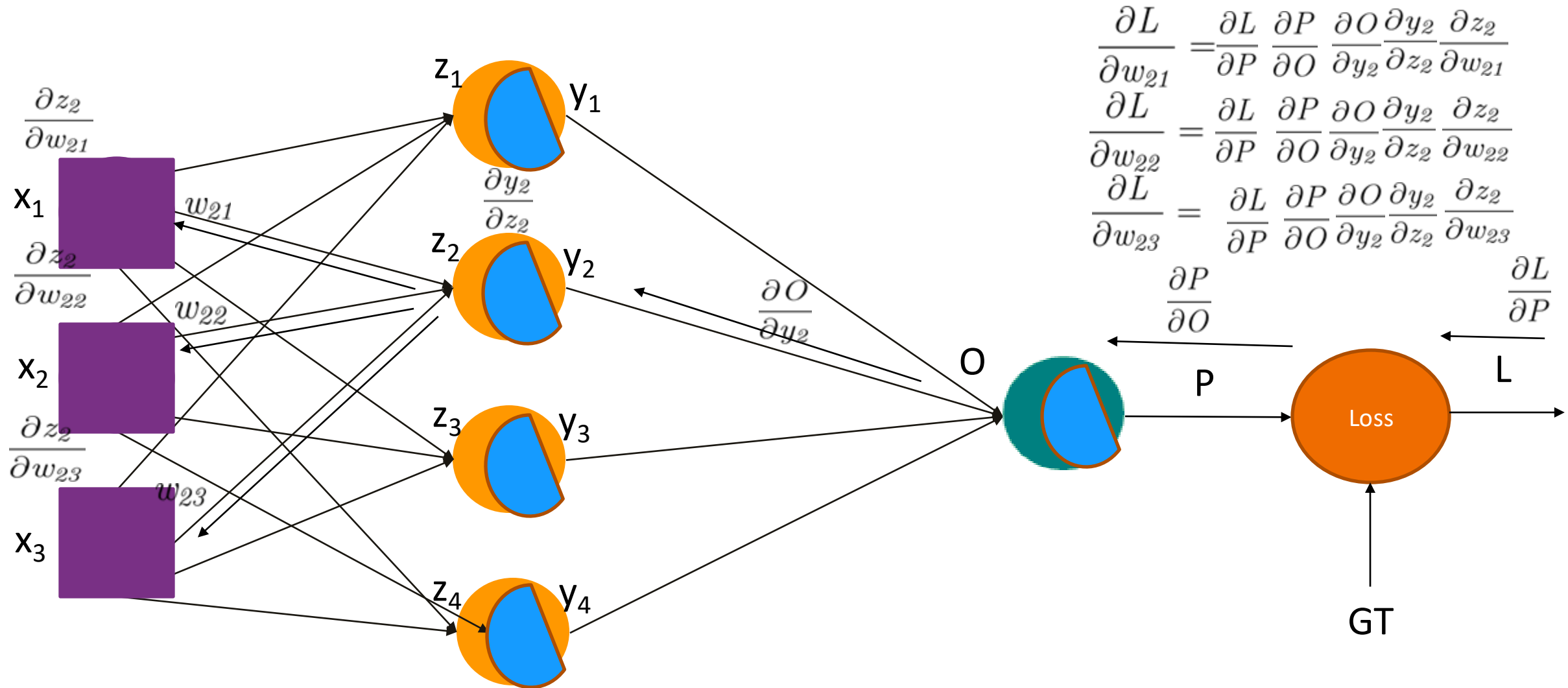
Detailed View: Back Propagation



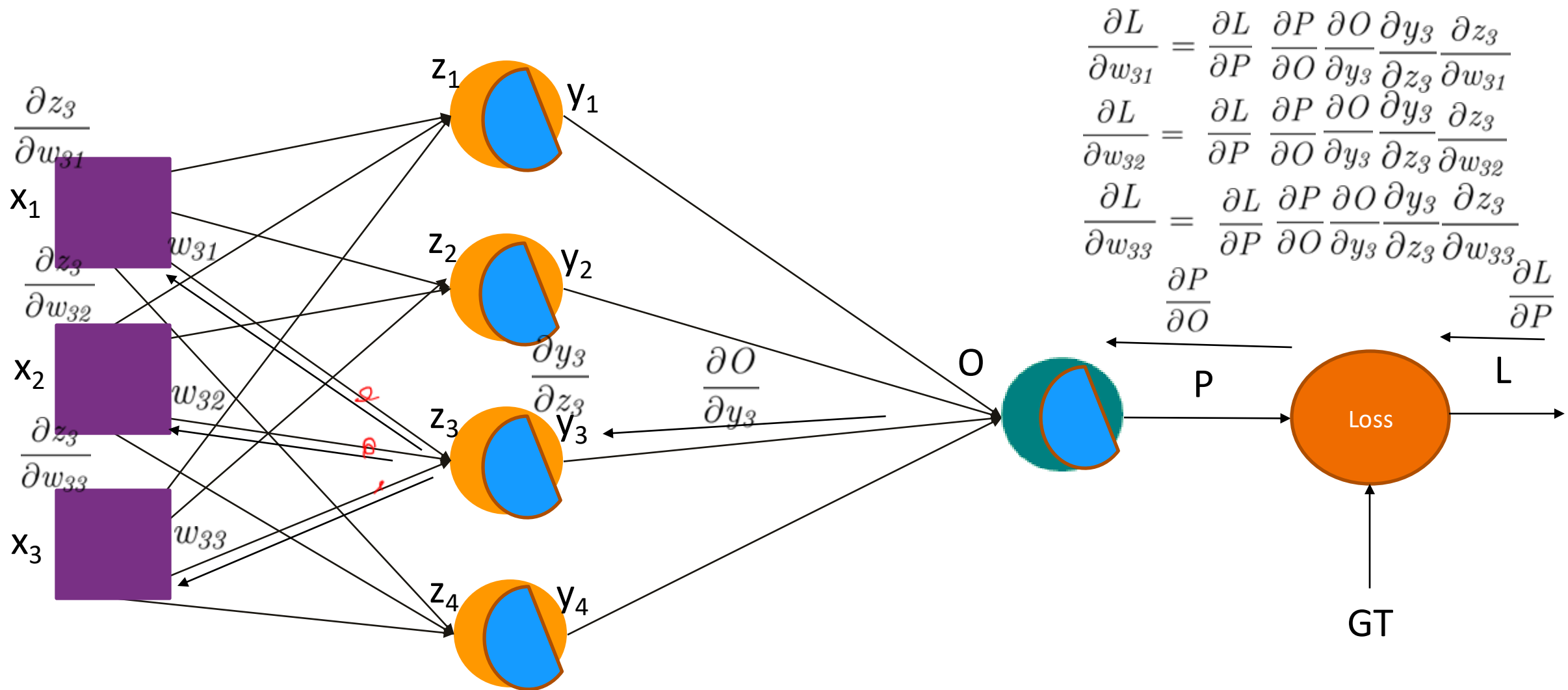
Detailed View: Back Propagation



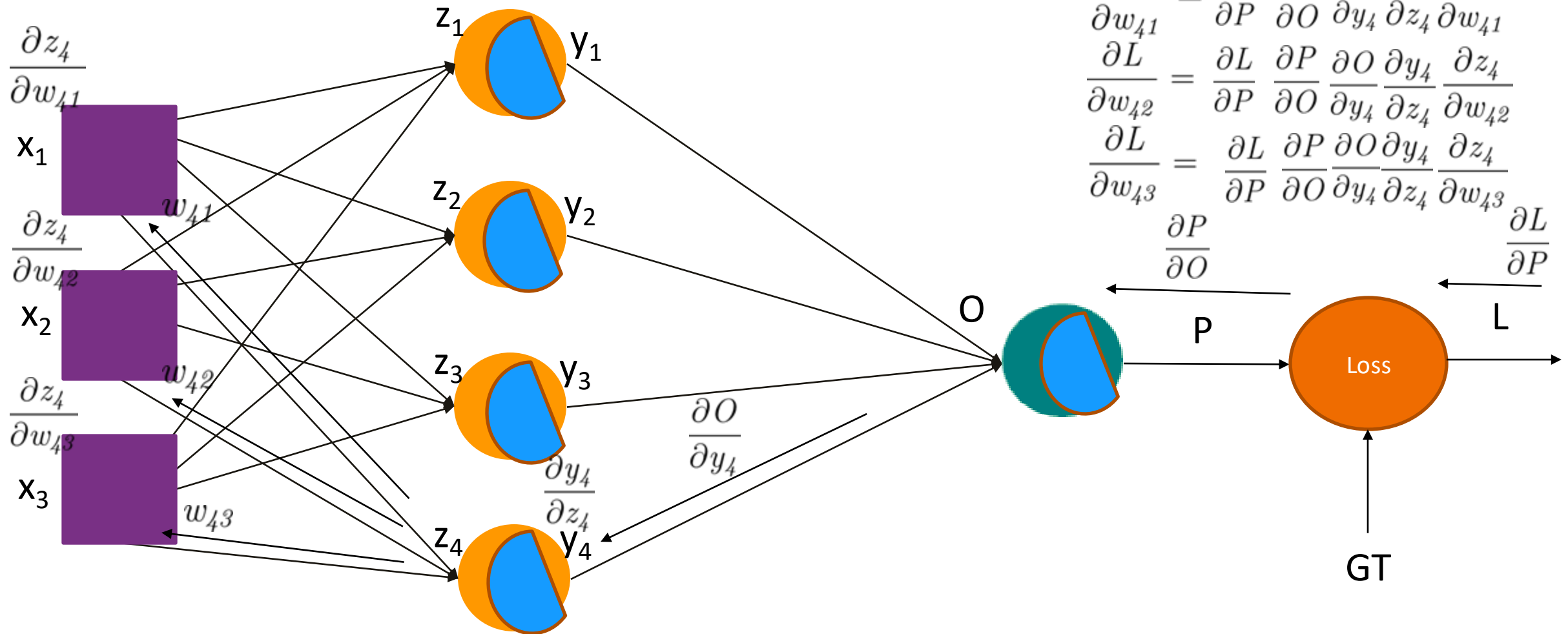
Detailed View: Back Propagation



Detailed View: Back Propagation



Detailed View: Back Propagation



BLANK SLIDE

Summary

- **Step 0:**
 - Initialize the Network (MLP), weights
- **Step 1:**
 - Do forward pass for a batch of randomly selected samples.
 - Predict outputs with the existing weights.
- **Step 2:**
 - Compute Loss for the set of samples.

Summary

- Step 3:
 - Update all the weights using gradient descent.

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

- Step 4:
 - Repeat all steps till the Loss is less than a threshold.

BLANK SLIDE

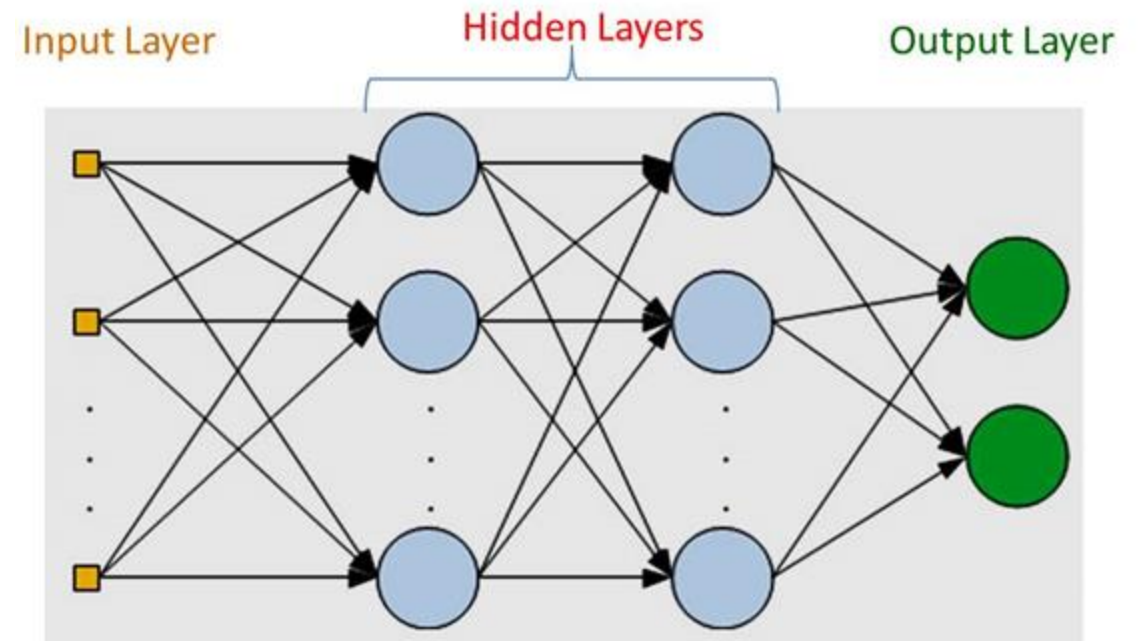
Back Propagation for MLP

- Two computational blocks/steps

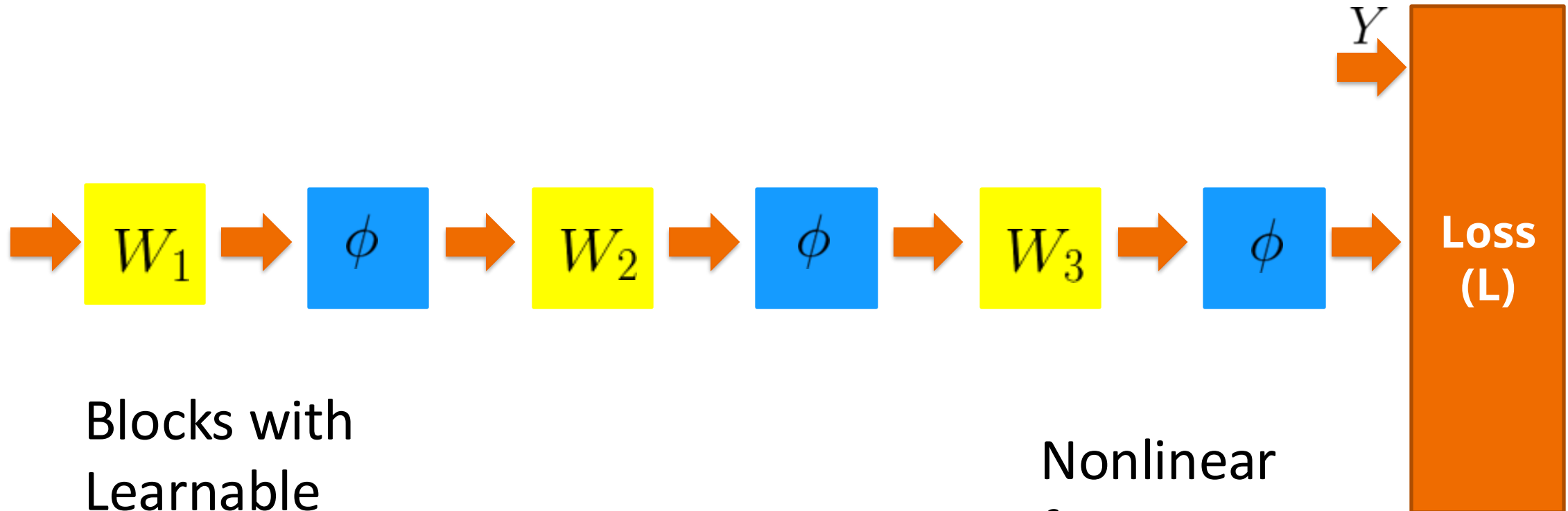
$$y = Wx$$

$$y = \phi(x) = \frac{1}{1 + e^{-x}}$$

- In either case we can compute $\frac{\partial y}{\partial x}$ easily.



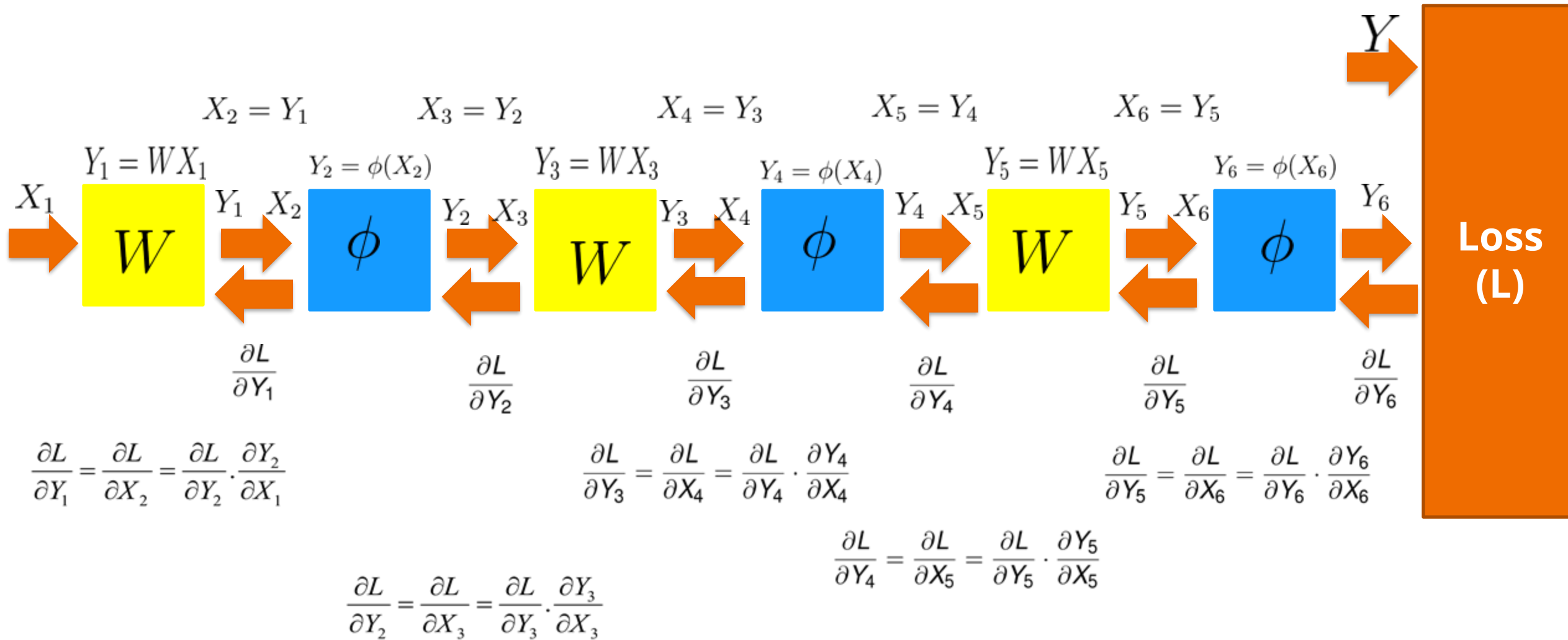
A simpler view point



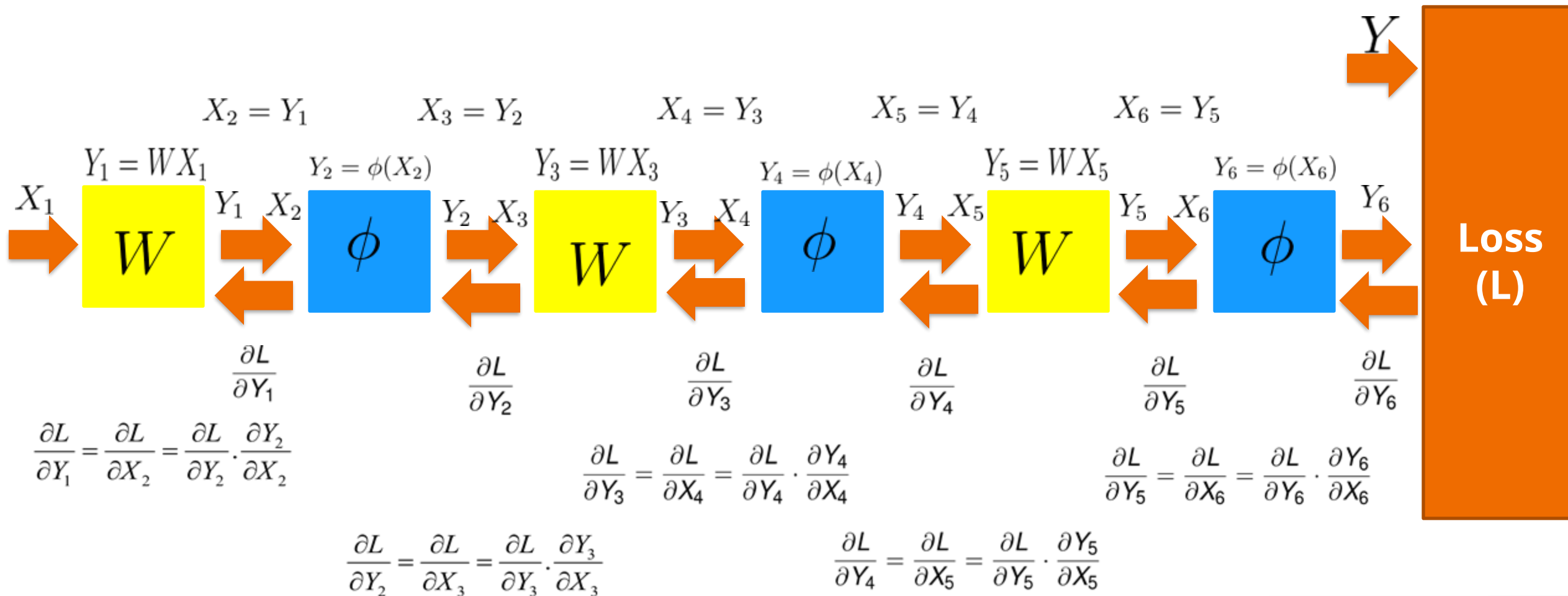
Blocks with
Learnable
parameters
Matrix
Multiplication

Nonlinear
functions
(often non learnable)

Back Propagation (X,Y): Propagation



Back Propagation (X,Y): Also Learning



$$W^{n+1} = W^n - \eta \frac{dL}{dW}$$

BLANK SLIDE

Back Propagation

$$(1) \rightarrow \frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x} \quad (2) \rightarrow \frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial W} \quad (3) \rightarrow W^{n+1} = W^n \eta \frac{\partial L}{\partial W}$$

- Let there be N stages. For a computational block ℓ ,
 - Compute $\frac{\partial L}{\partial x}$ using equation 1
 - If the block has learnable parameters $\mathbf{W}$ then,
 - Compute $\frac{\partial L}{\partial W}$ using equation 2
 - Update the parameters using equation 3
 - Set the $\frac{\partial L}{\partial x}$ of stage ℓ as $\frac{\partial L}{\partial y}$ of stage $\ell - 1$, and repeat the steps 1-3, until we reach the first block.

Summary

- **Step 0:**
 - Initialize the Network (MLP), weights
- **Step 1:**
 - Do forward pass for a batch of randomly selected samples.
 - Predict outputs with the existing weights.
- **Step 2:**
 - Compute Loss for the set of samples.

Summary

- Step 3:
 - Update all the weights using gradient descent.

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

- Step 4:
 - Repeat all steps till the Loss is less than a threshold.

Questions?

Comments on BP

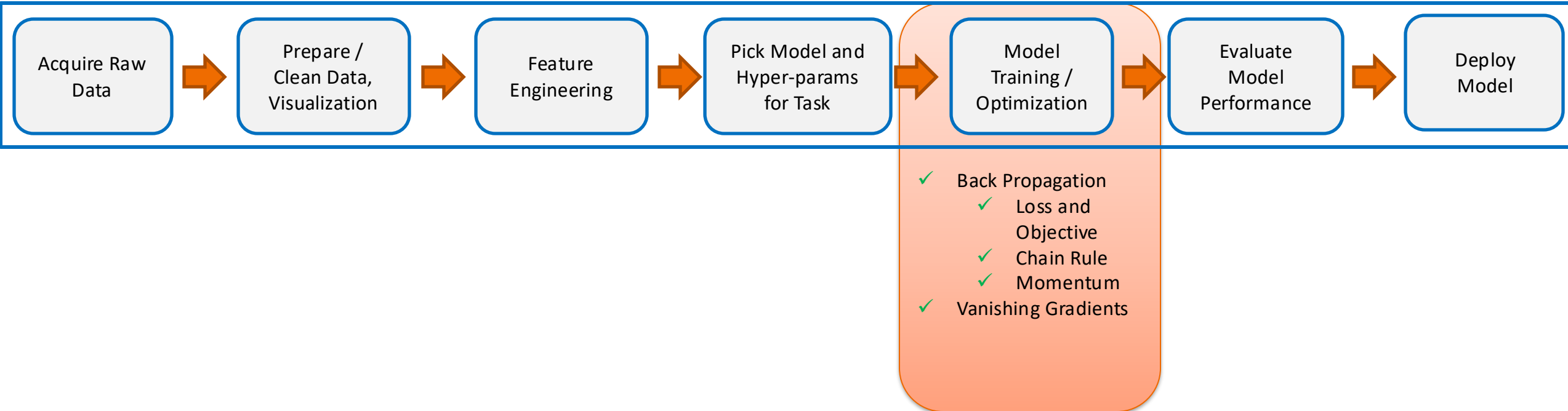
- A Non-Convex Optimization
 - Worry of getting trapped in Local Minima
- Why limit to simple gradient methods?
 - Why not second order methods?
- How to make the solution reliable?
 - Repeatable?
- Many refinements
 - Eg. Momentum
- **More Later**

Many Practical Challenges

- Vanishing Gradients in Deep Networks
- Slow Convergence
- Bad “Local Minimas”
- How to initialize the network?
 - Random or “smart” initializations
- Better update rule
 - Faster and reliable convergence
- Termination Criteria
 - When do we stop?

BLANK SLIDE

Summary



Thanks!!

Questions?